

LECTURE 15

EER Modeling: A Case Study

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- The case study's requirements, written the way a client would actually describe them
- Step-by-step identification of entity types, with the reasoning behind each decision
- Step-by-step identification of relationships and their structural constraints
- Step-by-step identification of attributes, including multi-valued and derived ones
- Building the staff specialization hierarchy — disjoint/overlapping, total/partial
- Applying aggregation and composition where the requirements genuinely call for them
- Assembling the complete EER diagram, layer by layer
- Validating the finished model against the original requirements, sentence by sentence

Case Study: Al-Shifa General Hospital

“The hospital administrator, in an initial requirements meeting”

We track every patient — basic details, every admission or out-patient visit. Every admission records ward, bed, admission/discharge dates.

We employ doctors, nurses, lab technicians, and admin staff — each has staffNo, name, CNIC, address, join date, and belongs to exactly one department. Doctors hold a license number and consultation fee. Some doctors are consultants (senior, own office); the rest are residents, supervised by a consultant.

Departments (Cardiology, Orthopedics, ...) have a name, location, and a head doctor. Every ward belongs to exactly one department; every bed to exactly one ward. A ward shut for renovation takes its beds with it.

Each admission has an attending doctor — responsible for that stay specifically. Patients also book appointments (with a confirmation number). A doctor may prescribe drugs during an admission or appointment, each with dosage/schedule. Every patient gets an invoice, and we track payment.

Step 1 — Identification of Entity Types

Applying the entity-vs-attribute test from Lecture 13 — own attributes, own relationships, independent existence?

Candidate	Entity?	Reasoning
Patient	Yes	Independent existence, own attributes, several relationships
Staff	Yes (superclass)	Shared attributes across every kind of employee
Doctor/Nurse/Technician/Administrator	Yes (subclass)	Role-specific attributes Staff alone can't hold cleanly
Consultant, Resident	Yes (2nd-level subclass)	Each has attributes the other doesn't
Department, Ward	Yes	Own name/location/head; ward owns beds
Bed	Yes — weak	Cannot be identified without knowing its ward
Admission, Appointment, Prescription	Yes	Own attributes (dates, confirmation no., dosage) — relationships promoted to entities
Drug, Invoice	Yes	Independent catalog / independent lifecycle
"Consultation fee", "CNIC"	No — attribute	Single values describing one entity, no attributes of their own

Step 2 — Identification of Relationships

Relationship	Participants	Cardinality	Notes
WorksIn	Staff, Department	N:1	Every staff member belongs to exactly one department
HeadedBy	Department, Doctor	1:1	Exactly one doctor heads a department
Supervises	Consultant, Resident	1:N	Unary via Doctor; a consultant supervises many residents
ComposedOf	Department, Ward	1:N	Composition – a ward can't exist outside its department
Owns	Ward, Bed	1:N	Bed is a weak entity, identified with its owning ward
AdmittedTo	Patient, Bed	N:1	Carries admissionDate/dischargeDate → promoted to Admission
AttendedBy	Admission (aggregate), Doctor	N:1	The attending doctor for a specific admission
Books / ScheduledWith	Patient-Appointment-Doctor	1:N / N:1	A patient books many appointments, each with one doctor
Prescribes / For / Of	Doctor-Prescription-Patient-Drug	1:N / N:1 / N:1	Ties a patient, doctor, and drug together
BilledTo	Patient, Invoice	1:N	A patient may accumulate several invoices over time

Step 3 — Identification of Attributes

Patient

patientNo
firstName
lastName
dateOfBirth
gender
address
phoneNumbers {}

Staff (before specialization)

staffNo
cnic
name
address
dateJoined
deptNo

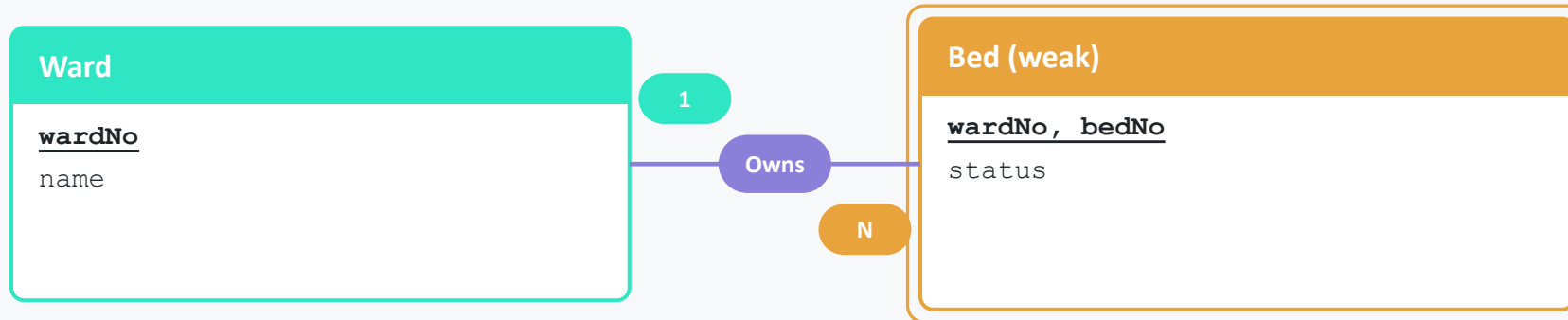
age is deliberately NOT stored (Lecture 13) — derived from dateOfBirth whenever needed rather than kept in sync by hand. phoneNumbers is genuinely multi-valued.

Remaining entity types' key structures (feeding Step 8):

Department(deptNo, name, location, headDoctorNo) · Ward(wardNo, name, deptNo) · Bed(wardNo, bedNo, status) · Admission(admissionNo, admissionDate, dischargeDate, reasonForAdmission, patientNo, wardNo, bedNo, attendingDoctorNo)

Step 4 — Bed: a Weak Entity

Bed cannot be uniquely identified on its own — a hospital with three wards each numbering beds 1, 2, 3... has no way to tell “Bed 3” in W1 apart from “Bed 3” in W2 using bedNo alone.

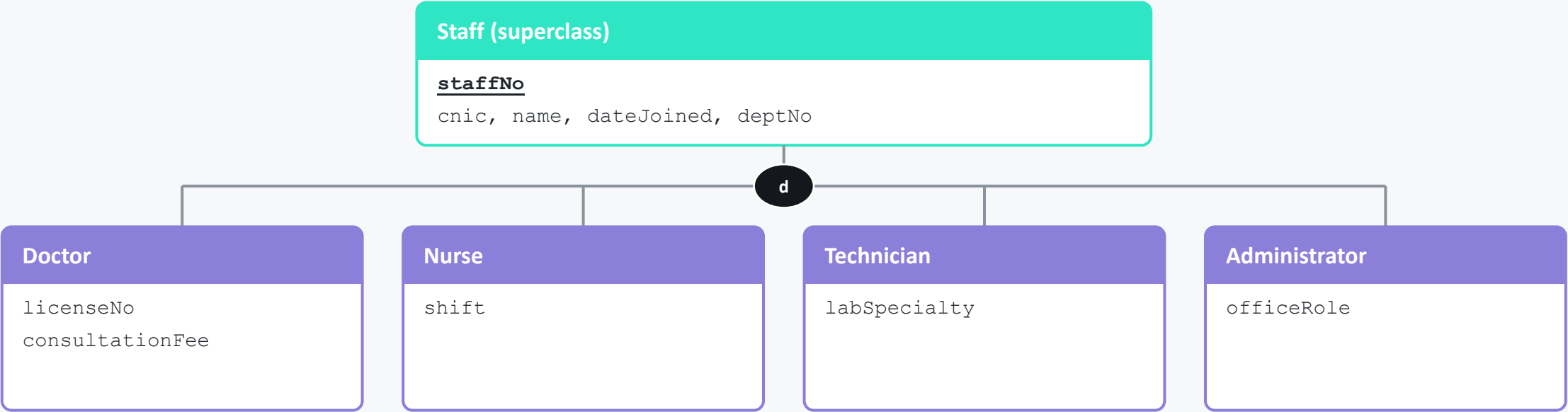


Weak entity: identified only in combination with its owner

Bed depends on Ward (its OWNER or identifying entity type) for existence AND identification. Its primary key is a COMPOSITE key made of its own partial key (bedNo) plus the ENTIRE primary key of its owner (wardNo) — shown here with the double-line border convention. This is the same pattern Lecture 16 maps directly into `Bed(wardNo, bedNo, status)` with ON DELETE CASCADE.

Step 5 — Level 1: Staff into Four Job Roles

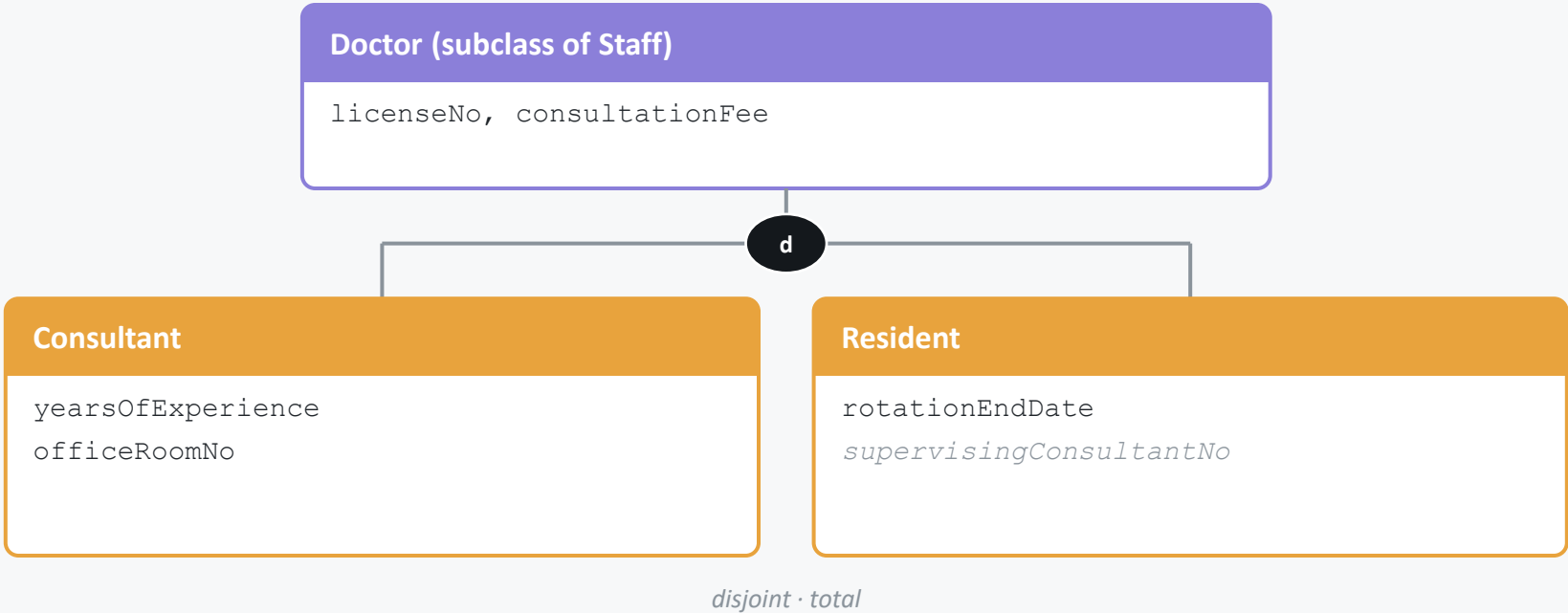
“We employ doctors, nurses, lab technicians, and administrative staff” — each role gets its own attributes, with no overlap mentioned anywhere.



Every staff member is described as being one of the four roles — total. No overlap between roles — disjoint.

Step 5 — Level 2: Doctor into Consultant / Resident

A second-level specialization, sitting directly on top of the Doctor subclass from Level 1. “Some doctors are consultants... the rest are residents” — again disjoint and total.



Resident.`supervisingConsultantNo` is a foreign key back into Consultant, capturing “residents... are supervised by one of the consultants” directly.

Why Two Separate Specialization Levels?

`Staff` → `Doctor` → {`Consultant`, `Resident`}, plus `Staff` → `Nurse`, `Technician`, `Administrator`

It would be POSSIBLE to flatten this into six disjoint leaves directly under Staff

Nurse, Technician, Administrator, Consultant, Resident, plain Doctor — skipping the intermediate Doctor level entirely.

That would lose real information: `licenseNo` and `consultationFee` genuinely belong to EVERY doctor, consultant or resident alike. Flattening would force duplicating those two attributes onto both leaf types instead of inheriting them once from a shared Doctor level.

The general check

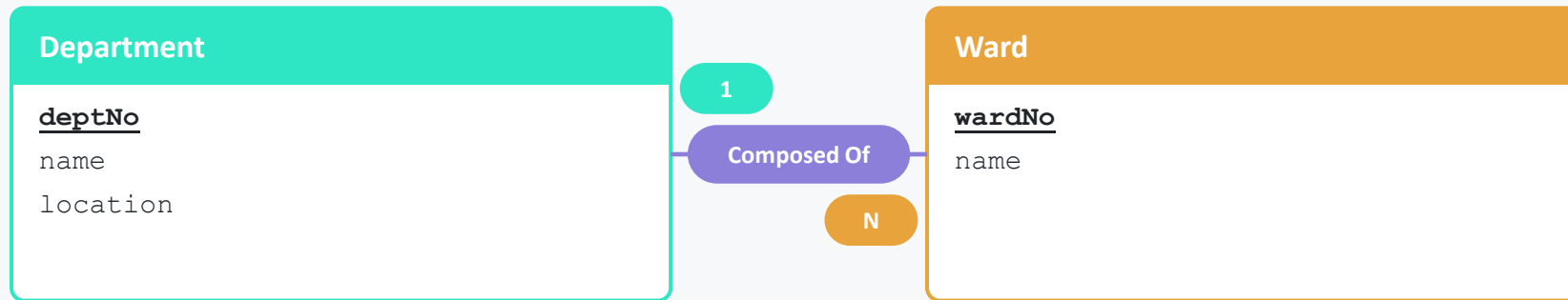
Multi-level specialization exists so shared structure at ANY depth gets factored out exactly once. Check for this the same way Step 1 checked for entities: attributes shared by some but not all immediate children of a node are a sign of a missing intermediate level.

Step 6 — Inheritance in This Hierarchy

- Every Doctor (and therefore every Consultant and Resident) inherits staffNo, cnic, name, address, dateJoined, deptNo from Staff, plus licenseNo and consultationFee from Doctor.
- Every Doctor also inherits Staff's relationships — WorksIn Department applies to a Consultant exactly as it does to a Nurse, with no separate relationship drawn per subclass.
- HeadedBy (a department's head must be a Doctor) and Supervises (only Consultants supervise, only Residents are supervised) belong specifically to Doctor, Consultant, and Resident.
- Nurse and Technician occurrences never participate in either — exactly the point of attaching a relationship at the subclass level rather than pushing it up to Staff.

Step 7 — Composition: Department “contains” Ward

“A ward that's shut down for renovation should take its beds with it out of the system entirely” — and a ward implicitly cannot outlive its department either. Strong ownership, no sharing, existence dependency: composition.

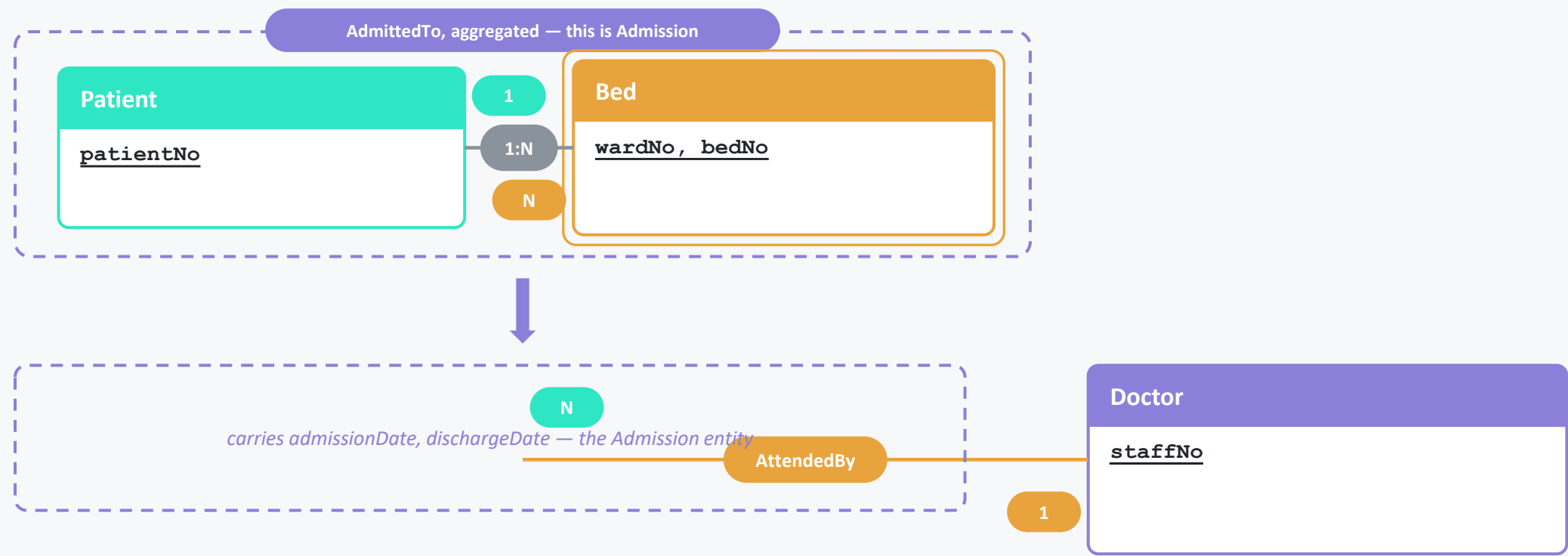


This is composition's defining signature

Wards are not shared between departments and have no independent existence once their department is gone. Composition, mapped in Lecture 16, becomes an ordinary 1:N foreign key placement PLUS ON DELETE CASCADE on Ward.deptNo — the strong-ownership fact traveling straight into the DDL.

Step 7 — Aggregation: Admission, Attended By a Doctor

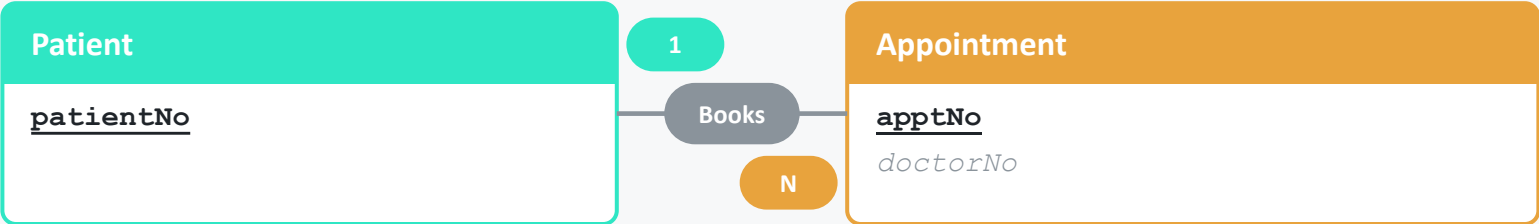
“The attending doctor — the doctor directly responsible for that patient's care during that stay, which might be different from any other doctor who happens to work on the same ward.”



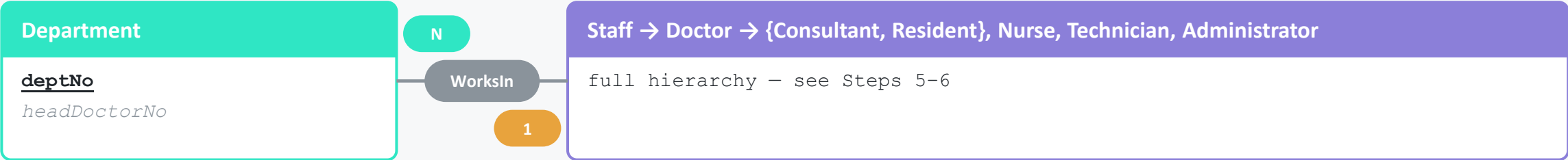
It's a fact about the specific admission (the Patient–Bed pairing as a whole) — not about the patient alone, nor the ward alone.

Step 8 — Assembling the EER Diagram (Layers 1–2)

Layer 1 — core independent entities and their direct relationships



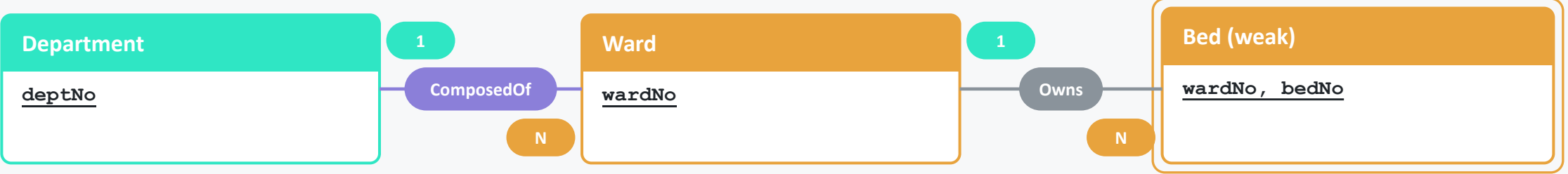
Layer 2 — the Staff specialization hierarchy (both levels), attached to Department



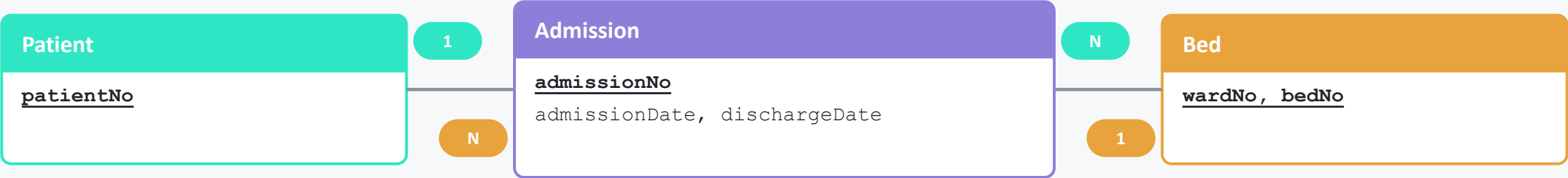
Each layer is a genuine, valid EER fragment on its own — deliberate. Building a large model one justified layer at a time, checking each against the requirements before moving on, is exactly how a real designer avoids the fan/chasm trap errors from Lecture 13.

Step 8 — Assembling the EER Diagram (Layers 3–4)

Layer 3 — Department, Ward, and Bed (composition + weak entity)



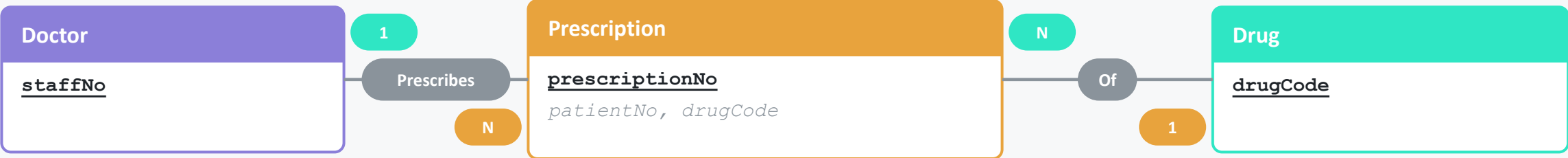
Layer 4 — Admission as an aggregate, attended by a Doctor



Layer 3 carries the ON DELETE CASCADE signature from composition; Layer 4 is the fully-resolved AdmittedTo aggregate from Step 7, ready for AttendedBy → Doctor on the next layer.

Step 8 — Layer 5 and the Comprehensive Model

Layer 5 — Prescription tying Patient, Doctor, and Drug together; Invoice per Patient



The comprehensive model — all five layers fused

Patient sits at the hub: Books → Appointment → ScheduledWith → Doctor; AdmittedTo → Bed (owned by Ward, composed under Department) → aggregated as Admission → AttendedBy → Doctor; For → Prescription → Of → Drug; BilledTo → Invoice. Doctor itself sits inside the two-level Staff specialization (→ Consultant/Resident), each Doctor WorksIn one Department, and one Doctor may be HeadedBy for that department.

Drawing all twenty-plus elements on one slide at once is exactly what Lecture 13 warned against — fan and chasm traps are far easier to introduce, and far harder to spot, when everything is drawn at once. Building layer-by-layer (Layers 1–5 above) and validating each against the requirements before fusing them is the safer, real-world workflow.

Step 9 — Validation of the EER Model

Requirement	Covered by
Track patients' basic details and admissions	Patient, Admission (aggregated AdmittedTo)
Admission records ward, bed, admission/discharge dates	Admission attrs; Bed (weak, owned by Ward)
4 staff roles, each with role attributes	Level 1 Staff specialization
Consultants have experience/office; residents rotate, supervised	Level 2 Doctor specialization; supervisingConsultantNo
Departments have name, location, head doctor	Department, HeadedBy
Wards belong to one department; removed with their department	ComposedOf – composition
Every bed belongs to exactly one ward	Owns – weak entity
Attending doctor is per-admission	AttendedBy on the aggregated Admission
Appointments have confirmation no., with a specific doctor	Appointment, ScheduledWith
Prescriptions tie patient, doctor, drug with dosage/schedule	Prescription entity
Every patient gets an invoice; payment tracked	Invoice, BilledTo

Assumptions Flagged Back to the Client

Validation means checking the model against the requirements in BOTH directions: every requirement is represented, and every model element traces back to a requirement.

1. Resident → exactly one supervising consultant

Assumed always exactly one, never zero and never more than one, at a time. The requirements imply this but don't state it in so many words.

2. Nurse / Technician: no further sub-specialization

The brief gives no role-specific detail beyond shift and labSpecialty. A reasonable place to stop — but a decision that should be confirmed with the client before treating it as final.

Flagging assumptions explicitly is itself part of validation

Rather than silently baking an assumption into the diagram, a real designer states it back to the client before implementation begins — the difference between a model that merely looks complete and one that is actually correct.

Key Takeaways

- A real EER design starts from ordinary client language, not a pre-sorted list of entities and relationships — the entity-vs-attribute and relationship-vs-entity tests from Lecture 13 do the actual sorting.
- Specialization can nest to more than one level (Staff → Doctor → {Consultant, Resident}) whenever a subset of an already-specialized subclass needs further differentiation.
- Weak entities (Bed), composition (Department/Ward), and aggregation (attended Admission) each solve a different, specific problem — match the construct to the sentence that motivates it.
- Validation checks the model against requirements in both directions, with every assumption stated explicitly rather than left implicit.